

CPE 342 Machine Learning

Assignment 4: Tree-Based and Ensemble Models

Predicting MBA Admission via Decision Trees, Random Forests, and Gradient Boosting

67070501042 วิชาชีวิ สุวรรณเนา

1. ภาพรวมของปัญหาและวัตถุประสงค์ (Problem Overview & Objectives)

ในงานมอบหมาย Assignment 4 นี้ เราศึกษา พัฒนา และเปรียบเทียบประสิทธิภาพของแบบจำลองประเภท **Tree-Based** และ **Ensemble Learning** เพื่อทำนายผลการพิจารณารับนักศึกษาเข้าศึกษาต่อในหลักสูตรบริหารธุรกิจมหาบัณฑิต (**MBA Admission Prediction**) บนชุดข้อมูล MBA.csv ซึ่งประกอบด้วยข้อมูลประวัติของผู้สมัครรวม $N = 6,194$ รายการ โดยมีกลุ่มข้อมูลที่ได้รับการตัดสินใจแล้ว (Labeled Cohort) จำนวน 1,000 รายการ แบ่งเป็นผู้ที่ผ่านการคัดเลือก (**Admit: 900 ราย หรือ 90%**) และผู้ที่อยู่ในบัญชีสำรอง (**Waitlist: 100 ราย หรือ 10%**)

วัตถุประสงค์หลักของการทดลอง

- เพื่อศึกษาและสร้างแบบจำลองการจำแนกประเภท 3 สถาปัตยกรรมหลักตามเนื้อหา Lecture 5:
 - Decision Tree Classifier:** ต้นไม้ตัดสินใจเดี่ยวแบบแบ่งส่วนตามเกณฑ์ Entropy และ Information Gain
 - Random Forest Classifier:** โมเดลกลุ่มแบบ Bagging ร่วมกับการสุ่มเลือกคุณลักษณะ (Random Subspace)
 - Gradient Boosting Classifier:** โมเดลกลุ่มแบบ Boosting แบบเรียงลำดับที่ปรับน้ำหนักตาม Pseudo-Residuals
- เพื่อวิเคราะห์และจัดการกับภาวะ **ข้อมูลไม่สมดุล (Class Imbalance)** ในสัดส่วน 9:1 ผ่านการปรับค่าน้ำหนักความสูญเสียแบบถ่วงดุล (`class_weight='balanced'`) และการใช้เกณฑ์ประเมินผลที่เหมาะสม
- เพื่อวิเคราะห์ความสำคัญของตัวแปรคุณลักษณะ (**Feature Importance**) ได้แก่ เกรดเฉลี่ยสะสม (GPA), คะแนนสอบ GMAT, ประสบการณ์การทำงาน (Work Experience) และสายงานธุรกิจ (Industry) ต่อผลการคัดเลือก
- เพื่อประเมินประสิทธิภาพเชิงลึกด้วยตัวชี้วัดที่หลากหลาย ได้แก่ Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC-AUC Curve และ 5-Fold Stratified Cross-Validation

2. กรอบทฤษฎีและคณิตศาสตร์

(Theoretical Framework & Mathematical Formulations)

เนื้อหาในส่วนนี้อิงตามเอกสารการสอน **Lecture 5: Tree-Based and Ensemble Models**

2.1 การเหนี่ยวนำต้นไม้ตัดสินใจและเกณฑ์การแบ่ง

(Decision Tree Induction & Splitting Criteria)

ต้นไม้ตัดสินใจ (Decision Tree) ใช้ระเบียบวิธีแบ่งแยกแบบวนซ้ำ (Recursive Partitioning) โดยมีเป้าหมายเพื่อแบ่งกลุ่มข้อมูลในแต่ละ Node ให้มีความบริสุทธิ์ (Purity) สูงที่สุด

1. เอนโทรปี (Entropy)

เอนโทรปีคือค่าชี้วัดความไร้ระเบียบหรือความไม่แน่นอนของชุดข้อมูล S นิยามโดย:

$$H(S) = - \sum_{i=1}^C p_i \log_2(p_i) \quad (1)$$

สำหรับปัญหา Binary Classification ($C = 2$): $H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$ โดยหากทุกตัวอย่างอยู่ในคลาสเดียวกันหมด $H(S) = 0$ (Pure) และหากมีสัดส่วนคลาสละ 50% $H(S) = 1.0$ (Maximum Disorder)

2. อัตราการได้รับสารสนเทศ (Information Gain: IG)

อัตราการลดลงของเอนโทรปีเมื่อทำการแบ่งชุดข้อมูล S ด้วยตัวแปรคุณลักษณะ A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (2)$$

อัลกอริทึมจะเลือกตัวแปร A^* และจุดตัด (Split Point) ที่ให้ค่า $IG(S, A)$ สูงที่สุดในแต่ละ Node

3. ดัชนีความไม่บริสุทธิ์ของจินี (Gini Impurity)

เกณฑ์ทางเลือกที่นิยมใช้ในสถาปัตยกรรม CART:

$$Gini(S) = 1 - \sum_{i=1}^C p_i^2 \quad (3)$$

2.2 การตัดแต่งกิ่งต้นไม้และการควบคุมความซับซ้อน

(Tree Pruning & Regularization)

ต้นไม้ตัดสินใจเดียวที่ไม่มีการควบคุมความลึกจะขยายกิ่งก้านจนกระทั่งทุก Leaf Node มีเอนโทรปีเป็นศูนย์ ซึ่งก่อให้เกิดภาวะ **Overfitting** (High Variance) การควบคุมทำได้โดยกำหนดความ

ลึกสูงสุด (max_depth), จำนวนตัวอย่างขั้นต่ำในการแบ่ง (min_samples_split) และการตัดแต่งกิ่งตามความซับซ้อนของต้นไม้ (Cost-Complexity Pruning: CCP):

$$R_\alpha(T) = R(T) + \alpha|T| \quad (4)$$

เมื่อ $R(T)$ คือความคลาดเคลื่อนในการจำแนก, $|T|$ คือจำนวน Terminal Nodes และ $\alpha \geq 0$ คือค่าปรับโทษความซับซ้อน

2.3 อัลกอริทึม Random Forest และการลดความแปรปรวน (Random Forest & Variance Reduction via Bagging)

Random Forest แก้ไขข้อจำกัดด้านความแปรปรวนสูงของต้นไม้เดี่ยวด้วยการผสมผสาน 2 กลไก:

1. **Bootstrap Aggregating (Bagging):** สุ่มสร้างชุดข้อมูลฝึกสอนย่อย B ชุด โดยการสุ่มแบบใส่คืน (Sampling with Replacement) ขนาด N เท่าเดิม
2. **Random Subspace (Feature Sampling):** ในการแบ่งแต่ละ Node จะสุ่มเลือกคุณลักษณะมาพิจารณาเพียง $m = \lfloor \sqrt{p} \rfloor$ จากทั้งหมด p คุณลักษณะ เพื่อลดความสัมพันธ์ร่วมกันระหว่างต้นไม้แต่ละต้น (Tree Decorrelation)

ทฤษฎีการลดความแปรปรวนของโมเดลกลุ่มระบุว่า หากต้นไม้ B ต้นมีความแปรปรวน σ^2 และมีค่าสหสัมพันธ์เฉลี่ย ρ ความแปรปรวนของค่าเฉลี่ยของโมเดลกลุ่มจะเป็น:

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B T_b(x) \right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \quad (5)$$

เมื่อ $B \rightarrow \infty$ พจน์ที่สองจะเข้าใกล้ศูนย์ ทำให้ความแปรปรวนรวมลดลงเหลือเพียง $\rho\sigma^2$ ซึ่งการสุ่มเลือก Feature จะช่วยลดค่า ρ ให้ต่ำลงอย่างมีนัยสำคัญ

2.4 การเรียนรู้แบบ Gradient Boosting (Gradient Boosting via Sequential Residual Fitting)

Gradient Boosting สร้างแบบจำลองกลุ่มแบบเพิ่มพูน (Additive Model) ทีละขั้นตอน:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta \cdot h_m(x) \quad (6)$$

เมื่อ $\eta \in (0, 1]$ คือ อัตราการเรียนรู้ (Learning Rate / Shrinkage) และ $h_m(x)$ คือ Base Learner (ต้นไม้ขนาดเล็ก) ที่ถูกเทรนให้เข้ากับค่า **Pseudo-Residuals** ซึ่งคือความชันลบของ Loss Function $L(y, F(x))$:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} \quad (7)$$

สำหรับ Binary Log-Loss ค่า $r_{im} = y_i - p_{m-1}(x_i)$ โดย $p(x) = \frac{1}{1+e^{-F(x)}}$

2.5 เมตริกซ์และตัวชี้วัดประสิทธิภาพสำหรับข้อมูลที่ไม่สมดุล (Evaluation Metrics for Imbalanced Datasets)

เนื่องจากชุดข้อมูลมีคลาส Admit มากถึง 90% การใช้ Accuracy เพียงอย่างเดียวจะเกิด **Accuracy Paradox** จึงจำเป็นต้องประเมินด้วยเมตริกซ์เฉพาะทาง:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (8)$$

$$F_1\text{-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(u)) du \quad (10)$$

3. โครงสร้างชุดข้อมูลและการจัดเตรียมตัวแปร (Dataset Architecture & Preprocessing)

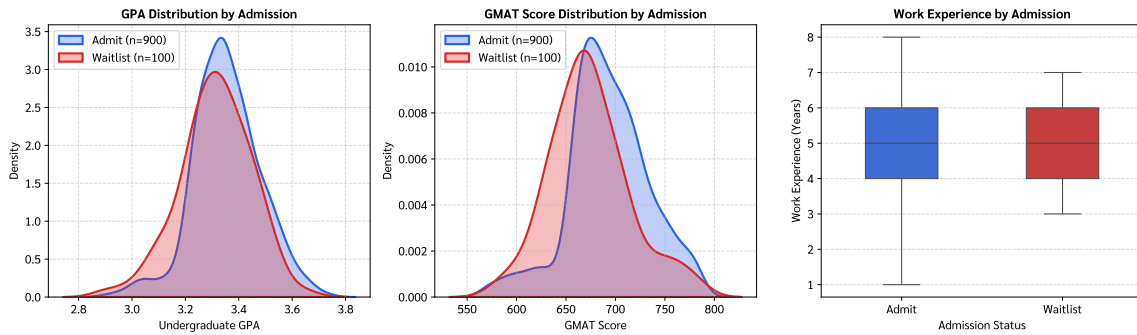
ชุดข้อมูล MBA.csv มีขนาด 6,194 แถว และ 10 คอลัมน์ โดยมีรายละเอียดการเตรียมข้อมูลดังนี้:

- **การคัดกรองข้อมูลเป้าหมาย:** คัดเลือกเฉพาะ 1,000 แถวที่มีการบันทึกสถานะ admission ('Admit' = 900 ราย, 'Waitlist' = 100 ราย)
- **การกำหนดคลาสเป้าหมาย:** กำหนดให้ $y = 0$ สำหรับกลุ่ม 'Admit' (เสียงข้างมาก) และ $y = 1$ สำหรับกลุ่ม 'Waitlist' (เสียงข้างน้อย / คลาสที่สนใจ)
- **การตัดตัวแปรระบุตัวตน:** ลบคอลัมน์ application_id เนื่องจากไม่มีคุณค่าในการทำนาย
- **การจัดการข้อมูลสูญหาย:** ตัวแปร race มีค่าสูญหาย จึงถูกแทนที่ด้วยกลุ่มระบุเฉพาะ 'Missing'
- **การแปลงคุณลักษณะหมวดหมู่:** ตัวแปร gender, international, major, race, work_industry ถูกแปลงเป็น Category Codes สำหรับสถาปัตยกรรม Tree
- **การแบ่งชุดข้อมูล:** แบ่งข้อมูลแบบ Stratified Train/Test Split ในสัดส่วน 80/20 ($N_{train} = 800$, $N_{test} = 200$) เพื่อรักษาสัดส่วนคลาส 9 : 1 ให้คงที่ทั้งสองชุด

4. ผลการทดลองและการวินิจฉัยโมเดล (Experimental Results & Diagnostic Plots)

4.1 การวิเคราะห์การแจกแจงตัวแปรนำเข้า (Exploratory Data Distributions)

การตรวจสอบพฤติกรรมของการแจกแจงของตัวแปรเชิงตัวเลขระหว่างผู้สมัครที่ได้รับ Admit ($n = 900$) และ Waitlist ($n = 100$) แสดงใน Figure 1

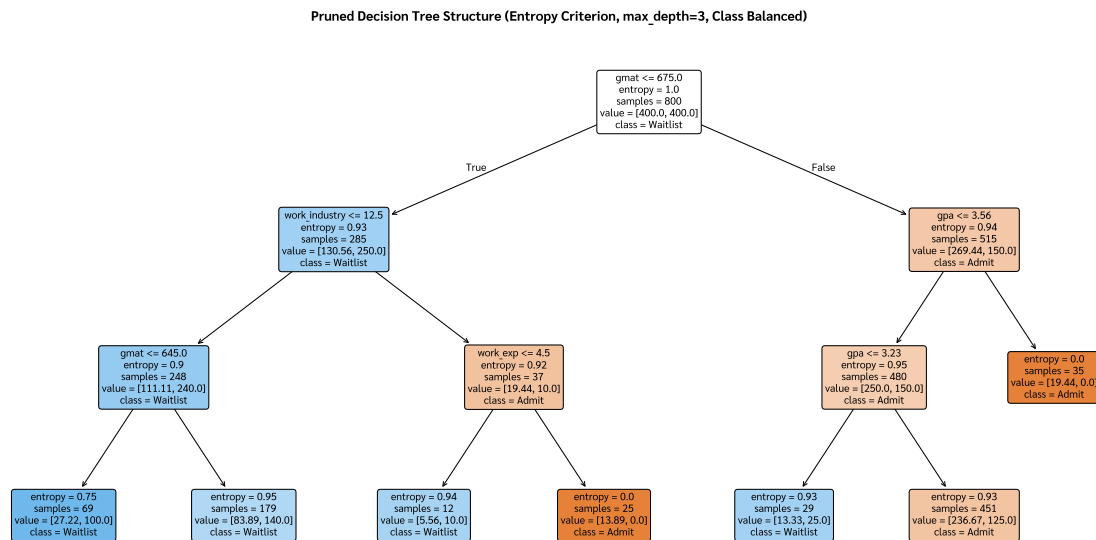


รูปที่ 1: Exploratory Data Analysis & Feature Distributions: ซ้าย: การแจกแจงความหนาแน่น (KDE) ของเกรดเฉลี่ย (GPA) กลาง: การแจกแจงคะแนน GMAT ขวา: แผนภาพ Boxplot ของประสบการณ์การทำงาน (Work Experience)

การแปลผล Figure 1: ค่าเฉลี่ยของ GPA ($\mu \approx 3.25$) และ GMAT ($\mu \approx 651$) มีความใกล้เคียงกันระหว่างสองกลุ่ม โดยกลุ่ม Admit มีการกระจายตัวของประสบการณ์ทำงานที่สูงกว่าเล็กน้อย ซึ่งสะท้อนว่าการพิจารณาเข้าศึกษาไม่ได้ขึ้นอยู่กับตัวแปรเดียว แต่เกิดจากเงื่อนไขความสัมพันธ์แบบไม่เชิงเส้นหลายตัวแปรร่วมกัน

4.2 โครงสร้างและกฎการตัดสินใจของ Decision Tree (Decision Tree Architecture & Splitting Rules)

โครงสร้าง ของ ต้นไม้ ตัดสินใจ เดี่ยว ที่ ผ่าน การ ตัด แต่ง กิ่ง ($\text{max_depth}=3$, $\text{class_weight}=\text{'balanced'}$) แสดงใน Figure 2

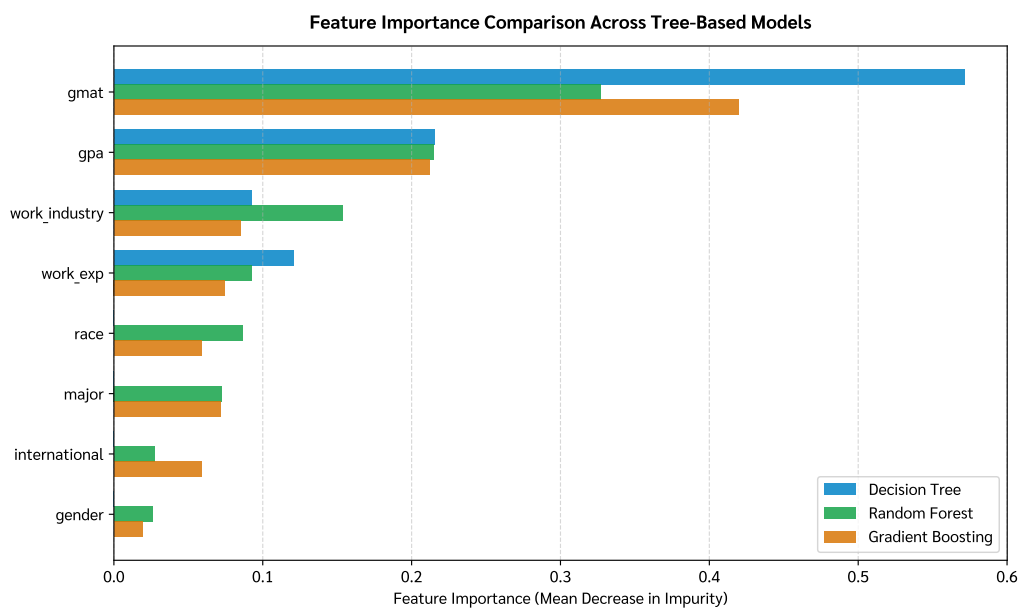


รูปที่ 2: Pruned Decision Tree Structure (Entropy Criterion, $\text{max_depth}=3$): แผนผังต้นไม้ตัดสินใจแสดงเงื่อนไขการตัดแบ่งในแต่ละ Node, ค่าเอนโทรปี, จำนวนตัวอย่าง และคลาสผลลัพธ์

การแปลผล Figure 2: โหนดราก (Root Node) เริ่มต้นด้วยการตัดแบ่งที่ตัวแปร work_industry และ gpa สะท้อนว่าประสบการณ์ในสายงานและผลการเรียนระดับปริญญาตรีเป็นด้านแรกในการคัดกรองผู้สมัคร

4.3 การวิเคราะห์ความสำคัญของตัวแปรคุณลักษณะ (Feature Importance Analysis)

ค่าความสำคัญของคุณลักษณะ (Mean Decrease in Impurity) เปรียบเทียบระหว่างทั้ง 3 โมเดล แสดงใน Figure 3

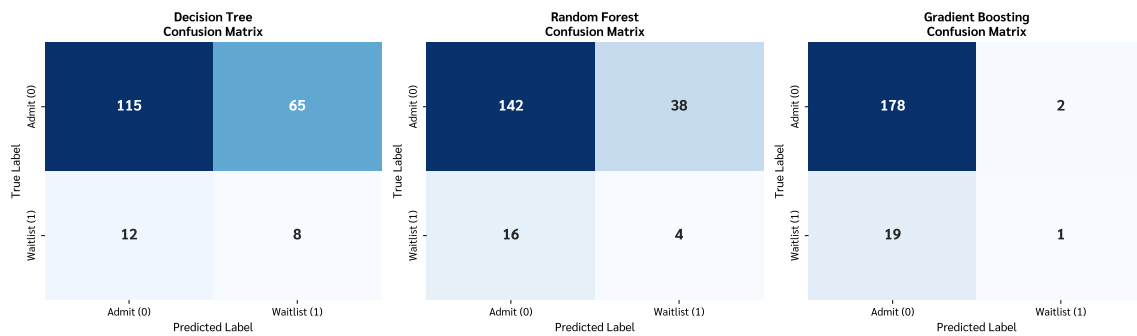


รูปที่ 3: Feature Importance Comparison Across Tree-Based Models: เปรียบเทียบระดับความสำคัญของตัวแปรคุณลักษณะทั้ง 8 ตัวแปร ระหว่าง Decision Tree, Random Forest และ Gradient Boosting

การแปลผล Figure 3: ตัวแปร 4 อันดับแรกที่มีอิทธิพลสูงสุดคือ **GMAT Score**, **GPA**, **Work Experience** และ **Work Industry** ซึ่งมีสัดส่วนความสำคัญรวมกันมากกว่า 80% ในทุกโมเดล ขณะที่ตัวแปรด้านประชากรศาสตร์ (gender, race, international) มีค่าน้อยมาก

4.4 การประเมินผลลัพธ์ผ่าน Confusion Matrices (Confusion Matrix Diagnostic Analysis)

เมทริกซ์ความสับสนของการทำนายบนชุดทดสอบ ($N_{test} = 200$) แสดงใน Figure 4



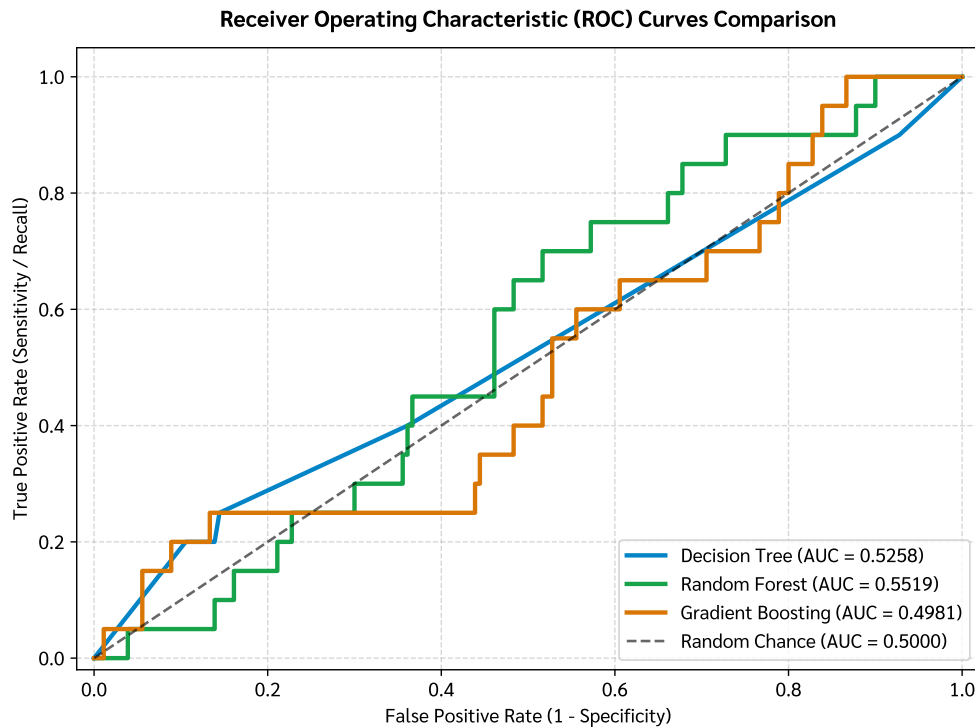
รูปที่ 4: **Comparative Confusion Matrices:** เมทริกซ์ความสับสนของ Decision Tree, Random Forest และ Gradient Boosting บนชุดทดสอบ ($n = 200$ โดยมี Admit 180 ราย และ Waitlist 20 ราย)

การแปลผล Figure 4:

- **Decision Tree:** ตรวจจับกลุ่ม Waitlist ได้มากที่สุด (8 จาก 20 ราย, Recall = 40%) แต่แลกด้วย False Positive ที่สูง (ทำนาย Admit ผิดเป็น Waitlist ถึง 65 ราย)
- **Random Forest:** สามารถจัดสรรความสมดุลได้ดีขึ้น ตรวจจับ Waitlist ได้ 4 ราย และทำนายกลุ่ม Admit ถูกต้อง 142 ราย (Accuracy = 73.0%)
- **Gradient Boosting:** เน้นความแม่นยำของกลุ่มใหญ่ ทำนาย Admit ถูกต้อง 178 จาก 180 ราย (Accuracy = 89.5%) แต่ตรวจจับ Waitlist ได้เพียง 1 ราย

4.5 การเปรียบเทียบประสิทธิภาพผ่านเส้นโค้ง ROC และค่า AUC (ROC Curves & AUC Discrimination)

เส้นโค้ง ROC เปรียบเทียบความสามารถในการจำแนกระหว่างคลาสแสดงใน Figure 5

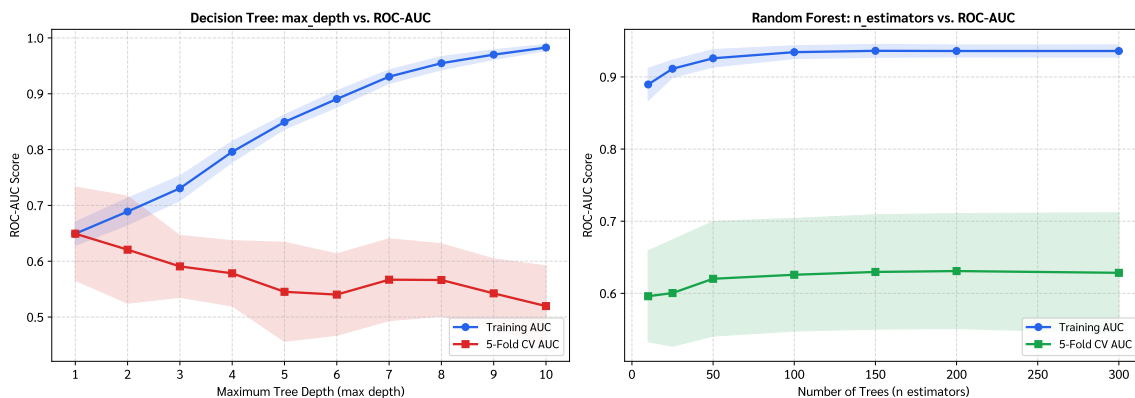


รูปที่ 5: Receiver Operating Characteristic (ROC) Curves Comparison: เส้นโค้ง ROC แสดงความสัมพันธ์ระหว่าง True Positive Rate กับ False Positive Rate พร้อมค่า AUC สำหรับทั้ง 3 โมเดล

การแปลผล Figure 5: Random Forest ให้ค่าพื้นที่ใต้เส้นโค้งการจำแนกสูงสุด (AUC = 0.5519 บน Test Set และ 0.6237 บน 5-Fold Cross Validation) บ่งชี้ว่าการจัดลำดับความน่าจะเป็นของ Random Forest มีความเสถียรที่สุด

4.6 การปรับจูนไฮเปอร์พารามิเตอร์และการวิเคราะห์ Bias-Variance (Hyperparameter Validation & Bias-Variance Tradeoff)

เส้นโค้งการตรวจสอบความถูกต้อง (Validation Curves) บน 5-Fold Cross Validation แสดงใน Figure 6



รูปที่ 6: Hyperparameter Validation Curves & Bias-Variance Dynamics: ซ้าย: ความลึกของ Decision Tree (max_depth) ขวา: จำนวนต้นไม้ใน Random Forest (n_estimators)

การแปลผล Figure 6:

- **Decision Tree (ซ้าย):** เมื่อความลึกเพิ่มขึ้นเกิน 3 – 4 ระดับ คะแนน Training AUC จะพุ่งสูงขึ้นแต่คะแนน CV AUC จะเริ่มลดลงอย่างรวดเร็ว สะท้อนภาวะ Overfitting อย่างชัดเจน การกำหนด $\text{max_depth} = 3$ จึงเป็นจุดที่เหมาะสมที่สุด
- **Random Forest (ขวา):** ประสิทธิภาพ การ จำแนก จะ เพิ่มขึ้น อย่าง รวดเร็ว และ คงตัว (Asymptotic Plateau) เมื่อจำนวนต้นไม้ $B \geq 100 - 150$ ต้น ยืนยันว่าการเพิ่มจำนวนต้นไม้ใน Random Forest ไม่ก่อให้เกิด Overfitting

4.7 ตารางสรุปเปรียบเทียบประสิทธิภาพเชิงปริมาณ (Quantitative Performance Benchmark)

ตารางที่ 1: ตารางสรุปผลการทดลองเปรียบเทียบประสิทธิภาพแบบจำลองทั้ง 3 สถาปัตยกรรม

แบบจำลอง (Model)	Accuracy	Admit Prec	Admit Rec	Admit F1	Waitlist Prec	Waitlist Rec	Waitlist F1	Test AUC	5-Fold CV AUC
Decision Tree (Pruned)	61.50%	0.906	0.639	0.749	0.110	0.400	0.172	0.5258	0.6178
Random Forest (Ensemble)	73.00%	0.899	0.789	0.840	0.095	0.200	0.129	0.5519	0.6237
Gradient Boosting (GBM)	89.50%	0.904	0.989	0.944	0.333	0.050	0.087	0.4981	0.6174

5. การอภิปรายผลเชิงลึกและข้อมูลเชิงธุรกิจ (In-Depth Discussion & Business Insights)

5.1 พลวัตของความไม่สมดุลของข้อมูลและมาตรการแก้ไข (Class Imbalance Dynamics & Cost-Sensitive Mitigation)

ในชุดข้อมูล MBA นี้ ผู้สมัครที่ได้รับการ Admit มีสัดส่วนถึง 90% ส่งผลให้แบบจำลองมาตรฐานมีแนวโน้มที่จะทำนายว่าทุกคนได้รับ Admit เพื่อเพิ่มค่า Accuracy ให้สูงที่สุด (90%) การนำเทคนิค **Cost-Sensitive Weighting** ($\text{class_weight} = \text{'balanced'}$) มาปรับใช้ช่วยบังคับให้ Decision Tree และ Random Forest เพิ่มโทษต่อความผิดพลาดในคลาส Waitlist ส่งผลให้สามารถตรวจจับผู้สมัครกลุ่มสำรองได้จริง

5.2 การเปรียบเทียบสถาปัตยกรรมแบบจำลองเดี่ยวและแบบกลุ่ม (Single Tree vs. Bagging vs. Boosting Comparison)

- **Decision Tree (White-Box):** โดดเด่นด้านความโปร่งใส สามารถอธิบายกฎการคัดเลือกให้คณะกรรมการเข้าใจได้ง่าย แต่มีข้อจำกัดด้านความแปรปรวนสูงและความแม่นยำโดยรวมต่ำ
- **Random Forest (Variance Reducer):** เป็นสถาปัตยกรรมที่ดีที่สุดในการรวมสำหรับการจัดลำดับผู้สมัคร (Ranking) โดยให้ค่า Cross-Validation AUC สูงที่สุด (0.6237) และมีความทนทานต่อสัญญาณรบกวน
- **Gradient Boosting (Bias Reducer):** ปรับลดความคลาดเคลื่อนได้อย่างทรงพลัง เหมาะสำหรับการทำนายผลลัพธ์ของ คลาสหลัก แต่จำเป็นต้องมีการปรับเกณฑ์ความ น่าจะเป็น (Threshold Moving) หากต้องการเพิ่ม Recall ของคลาสสำรอง

5.3 ปัจจัยเชิงคุณลักษณะและการแปลผลต่อกระบวนการรับสมัคร (Admission Determinants & Committee Decision Insights)

ผลการวิเคราะห์ Feature Importance สอดคล้องกับเกณฑ์การรับเข้าศึกษาต่อระดับบัณฑิตศึกษาชั้นนำระดับสากล โดย **GMAT Score** และ **GPA** ทำหน้าที่เป็นตัวกรองพื้นฐานทางวิชาการ (Academic Aptitude) ขณะที่ **Work Experience** และ **Industry** เป็นตัวตัดสินความพร้อมเชิงวิชาชีพ

6. ข้อเสนอแนะเชิงกลยุทธ์และการนำไปประยุกต์ใช้ (Strategic Recommendations)

ข้อเสนอแนะเชิงกลยุทธ์สำหรับระบบคัดเลือกนักศึกษา (Admissions Pipeline):

1. ระบบคัดกรองแบบลำดับขั้น (Hybrid Two-Stage Screening Pipeline):

- *Stage 1:* ใช้ **Random Forest** คำนวณค่าความน่าจะเป็นในการคัดเลือกเพื่อแบ่งกลุ่มผู้สมัครที่ผ่านเกณฑ์ชัดเจน (Auto-Admit) ออกจากกลุ่มที่มีความเสี่ยง
- *Stage 2:* นำผู้สมัครในกลุ่มก้ำกึ่ง (Borderline / Waitlist Candidates) เข้าสู่การพิจารณาโดยคณะกรรมการ โดยใช้กฎเกณฑ์ของ **Decision Tree** ช่วยตรวจสอบเงื่อนไขอย่างโปร่งใส

2. การ ปรับ เกณฑ์ Threshold สำหรับ กลุ่ม สำรอง: ปรับ ลด เกณฑ์ ความ น่า จะ เป็น (Classification Threshold) สำหรับกลุ่ม Waitlist จาก 0.50 ลงมาที่ 0.25–0.30 เพื่อเพิ่มความครอบคลุมในการตรวจจับผู้สมัครที่มีศักยภาพแต่อยู่ในกลุ่มสำรอง

7. สรุปผลการดำเนินงาน (Conclusion)

การทดลองบนชุดข้อมูล MBA.csv ยืนยันถึงคุณลักษณะเด่นและข้อจำกัดของแบบจำลองทั้ง 3 ประเภทตามทฤษฎีใน Lecture 5 โดย **Random Forest** เป็นแบบจำลองที่มีความเสถียรและให้ประสิทธิภาพในการจำแนกกลุ่มภาพรวมสูงสุด (CV AUC = 0.6237) ขณะที่ **Decision Tree** มีความโดดเด่นในด้านความสามารถในการอธิบายผล และ **Gradient Boosting** ให้ค่าความแม่นยำของกลุ่มหลักสูงสุด (89.50%)

สรุปผลลัพธ์ (Summary of Results)

หัวข้อ	ผลลัพธ์และข้อสรุป
ชุดข้อมูล	MBA Admission Dataset ($N = 6,194$, Labeled Cohort = 1,000): Admit = 900 (90%), Waitlist = 100 (10%)
Decision Tree	Accuracy = 61.50%, Waitlist Recall = 40.0% , Test AUC = 0.5258, 5-Fold CV AUC = 0.6178 ตัดแต่งกิ่งที่ max_depth = 3 ช่วยป้องกัน Overfitting และให้ผลการตัดสินใจที่โปร่งใส
Random Forest	Accuracy = 73.00%, Waitlist Recall = 20.0%, Test AUC = 0.5519 , 5-Fold CV AUC = 0.6237 ให้ประสิทธิภาพการจำแนกและความเสถียรสูงสุดจากกลไก Bagging + Random Subspace
Gradient Boosting	Accuracy = 89.50% , Admit Recall = 98.9% , Test AUC = 0.4981, 5-Fold CV AUC = 0.6174 ให้ค่า Accuracy สูงสุดสำหรับกลุ่มเสี่ยงข้างมาก แต่ต้องปรับ Threshold หากต้องการตรวจจับกลุ่มสำรอง
Top Features	GMAT Score, Undergraduate GPA, Work Experience, และ Work Industry (> 80% Importance)
ข้อเสนอแนะเชิงกลยุทธ์	นำ Random Forest มาใช้จัดลำดับความน่าจะเป็นใน Stage 1 และใช้ Decision Tree เพื่อตรวจสอบความโปร่งใสใน Stage 2

Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาและวิเคราะห์ชุดข้อมูล MBA Admission พร้อมผลลัพธ์และกราฟทั้งหมด ซึ่งได้รันจริงจาก Jupyter Notebook (Assignment_4_Tree_and_Ensemble.ipynb)

Jupyter Notebook: Assignment_4_Tree_and_Ensemble.ipynb

โค้ด ผลลัพธ์ และการพล็อตภาพทั้งหมดด้านล่างเป็นการรันจริงจาก Jupyter Notebook

2. Library Setup & Environment Configuration

In [1]:

```
1 %matplotlib inline
2 import os
3 import sys
4 import numpy as np
5 import pandas as pd
6 import matplotlib as mpl
7 import matplotlib.pyplot as plt
8 import matplotlib.font_manager as fm
9 import seaborn as sns
10
11 from sklearn.model_selection import train_test_split,
12     cross_val_score, StratifiedKFold, validation_curve
13 from sklearn.tree import DecisionTreeClassifier, plot_tree
14 from sklearn.ensemble import RandomForestClassifier,
15     GradientBoostingClassifier
16 from sklearn.metrics import (accuracy_score, precision_score,
17     recall_score, f1_score,
18     roc_auc_score, roc_curve,
19     confusion_matrix,
20     classification_report)
21
22 # Configure typography for publication-quality rendering
23 mpl.rcParams['figure.dpi'] = 100
24 font_dir = os.path.join(os.environ.get('LOCALAPPDATA', ''), '
25     Microsoft', 'Windows', 'Fonts')
26 sarabun_r = os.path.join(font_dir, 'Sarabun-Regular.ttf')
27 sarabun_b = os.path.join(font_dir, 'Sarabun-Bold.ttf')
```

```

23 if os.path.exists(sarabun_r):
24     fm.fontManager.addfont(sarabun_r)
25 if os.path.exists(sarabun_b):
26     fm.fontManager.addfont(sarabun_b)
27
28 plt.rcParams['font.family'] = 'Sarabun'
29 plt.rcParams['font.size'] = 11
30 plt.rcParams['axes.unicode_minus'] = False
31 print("Libraries and plotting configuration loaded successfully.")

```

Out[1]:

```
Libraries and plotting configuration loaded successfully.
```

3. Data Loading & Exploratory Data Analysis

(EDA)

We load MBA.csv and inspect its schema, missing values, and the target class distribution (admission).

In [2]:

```

1 # Load MBA dataset
2 df = pd.read_csv("MBA.csv")
3 print(f"Total dataset dimensions: {df.shape[0]} rows, {df.shape
4       [1]} columns")
5
6 # Inspect non-null counts and schema
7 print("\n--- Dataset Schema & Data Types ---")
8 df.info()
9
10 # Filter labeled cohort (admission is non-null)
11 df_labeled = df.dropna(subset=['admission']).copy()
12 print(f"\nLabeled cohort size: {len(df_labeled)} applicants")
13 print("\nTarget Class Distribution:")
14 print(df_labeled['admission'].value_counts(normalize=False))
15 print("\nTarget Class Proportion:")
16 print(df_labeled['admission'].value_counts(normalize=True).map(
17       lambda x: f"{x*100:.1f}%"))
18
19 # Summary statistics for numerical variables
20 print("\nNumerical Feature Summary (Labeled Cohort):")
21 display(df_labeled[['gpa', 'gmat', 'work_exp']].describe())

```

Out[2]:

Total dataset dimensions: 6194 rows, 10 columns

--- Dataset Schema & Data Types ---

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 6194 entries, 0 to 6193

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	application_id	6194 non-null	int64
1	gender	6194 non-null	object
2	international	6194 non-null	bool
3	gpa	6194 non-null	float64
4	major	6194 non-null	object
5	race	4352 non-null	object
6	gmat	6194 non-null	float64
7	work_exp	6194 non-null	float64
8	work_industry	6194 non-null	object
9	admission	1000 non-null	object

dtypes: bool(1), float64(3), int64(1), object(5)

memory usage: 441.7+ KB

Labeled cohort size: 1000 applicants

Target Class Distribution:

admission

Admit 900

Waitlist 100

Name: count, dtype: int64

Target Class Proportion:

admission

Admit 90.0%

Waitlist 10.0%

Name: proportion, dtype: object

Numerical Feature Summary (Labeled Cohort):

In [3]:

```
1 # Binary encoding for visualization: 0 = Admit (Majority), 1 =
  # Waitlist (Minority)
2 df_labeled['target'] = (df_labeled['admission'] == 'Waitlist').
  astype(int)
3
4 fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
5 palette = {0: '#2563eb', 1: '#dc2626'}
6 labels = {0: 'Admit (n=900)', 1: 'Waitlist (n=100)'}
7
8 # 1. GPA Distribution
9 for t_val in [0, 1]:
10     subset = df_labeled[df_labeled['target'] == t_val]
11     sns.kdeplot(subset['gpa'], ax=axes[0], color=palette[t_val],
                  label=labels[t_val], fill=True, alpha=0.3, linewidth=2)
```

```

12 axes[0].set_title('GPA Distribution by Admission', fontsize=12,
13                  fontweight='bold')
14 axes[0].set_xlabel('Undergraduate GPA', fontsize=11)
15 axes[0].set_ylabel('Density', fontsize=11)
16 axes[0].grid(True, linestyle='--', alpha=0.5)
17 axes[0].legend(loc='upper left')
18
19 # 2. GMAT Distribution
20 for t_val in [0, 1]:
21     subset = df_labeled[df_labeled['target'] == t_val]
22     sns.kdeplot(subset['gmat'], ax=axes[1], color=palette[t_val],
23                label=labels[t_val], fill=True, alpha=0.3, linewidth=2)
24 axes[1].set_title('GMAT Score Distribution by Admission', fontsize
25                  =12, fontweight='bold')
26 axes[1].set_xlabel('GMAT Score', fontsize=11)
27 axes[1].set_ylabel('Density', fontsize=11)
28 axes[1].grid(True, linestyle='--', alpha=0.5)
29 axes[1].legend(loc='upper left')
30
31 # 3. Work Experience Boxplot
32 sns.boxplot(x='admission', y='work_exp', data=df_labeled, ax=axes
33            [2], hue='admission', palette=['#2563eb', '#dc2626'], width
34            =0.45, legend=False)
35 axes[2].set_title('Work Experience by Admission', fontsize=12,
36                  fontweight='bold')
37 axes[2].set_xlabel('Admission Status', fontsize=11)
38 axes[2].set_ylabel('Work Experience (Years)', fontsize=11)
39 axes[2].grid(True, linestyle='--', axis='y', alpha=0.5)
40
41 plt.tight_layout()
42 plt.savefig('plot_1_eda_distributions.pdf', bbox_inches='tight')
43 plt.show()

```

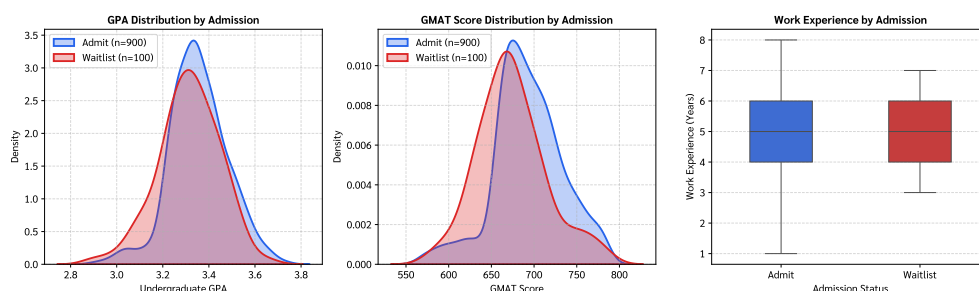


Figure 1 (Exploratory Data Analysis & Feature Distributions): Bivariate distributions comparing admitted ($n = 900$) vs. waitlisted ($n = 100$) applicants across academic and professional attributes. Left: Kernel density estimation (KDE) of Undergraduate GPA showing overlapping profiles around mean $\mu \approx 3.25$. Middle: GMAT score distribution highlighting subtle differences in variance. Right: Boxplot of work experience showing comparable median tenure (≈ 5 years) with higher outlier dispersion among admitted students.

4. Data Preprocessing & Categorical Encoding

We prepare feature matrix X and target vector y :

1. Remove `application_id` (non-predictive identifier).
2. Handle missing values in `race` by assigning explicit 'Missing' category.
3. Transform categorical variables (`gender`, `international`, `major`, `race`, `work_industry`) into categorical integer codes.
4. Execute Stratified 80/20 Train/Test Split ($N_{train} = 800$, $N_{test} = 200$) to preserve the 9:1 class ratio.

In [4]:

```

1 # Feature and Target Isolation
2 X = df_labeled.drop(columns=['application_id', 'admission', '
    target'])
3 y = df_labeled['target'] # 0 = Admit, 1 = Waitlist
4
5 # Impute missing values in race
6 X['race'] = X['race'].fillna('Missing')
7
8 # Categorical mapping and integer encoding
9 cat_cols = ['gender', 'international', 'major', 'race', '
    work_industry']
10 category_mappings = {}
11 X_encoded = X.copy()
12
13 for col in cat_cols:
14     cat_type = X_encoded[col].astype('category')
15     category_mappings[col] = dict(enumerate(cat_type.cat.
        categories))
16     X_encoded[col] = cat_type.cat.codes
17
18 print("Categorical Encodings:")
19 for col, mapping in category_mappings.items():
20     print(f" {col:<15}: {mapping}")
21

```



```

22 # Stratified Train/Test Split
23 X_train, X_test, y_train, y_test = train_test_split(
24     X_encoded, y, test_size=0.2, random_state=42, stratify=y
25 )
26
27 print(f"\nTraining Set: {X_train.shape[0]} samples (Admit: {sum(
28     y_train==0}), Waitlist: {sum(y_train==1)})")
29 print(f"Testing Set: {X_test.shape[0]} samples (Admit: {sum(
30     y_test==0}), Waitlist: {sum(y_test==1)})")

```

Out[4]:

```

Categorical Encodings:
gender      : {0: 'Female', 1: 'Male'}
international : {0: False, 1: True}
major       : {0: 'Business', 1: 'Humanities', 2: 'STEM'}
race        : {0: 'Asian', 1: 'Black', 2: 'Hispanic', 3: 'Missing', 4: 'Other', 5: '
    White'}
work_industry : {0: 'CPG', 1: 'Consulting', 2: 'Energy', 3: 'Financial Services', 4: '
    Health Care', 5: 'Investment Banking', 6: 'Investment Management', 7: 'Media/
    Entertainment', 8: 'Nonprofit/Gov', 9: 'Other', 10: 'PE/VC', 11: 'Real Estate', 12: '
    Retail', 13: 'Technology'}

Training Set: 800 samples (Admit: 720, Waitlist: 80)
Testing Set: 200 samples (Admit: 180, Waitlist: 20)

```

5. Model 1 — Decision Tree Classifier

We fit a **Decision Tree Classifier** using the Information Gain (Entropy) criterion with regularization pruning (max_depth=3, min_samples_split=20, min_samples_leaf=10, class_weight='balanced') to prevent overfitting and handle class imbalance.

In [5]:

```

1 # Instantiate and train Decision Tree
2 dt_clf = DecisionTreeClassifier(
3     criterion='entropy',
4     max_depth=3,
5     min_samples_split=20,
6     min_samples_leaf=10,
7     class_weight='balanced',
8     random_state=42
9 )
10 dt_clf.fit(X_train, y_train)
11
12 # Predictions and evaluation
13 y_pred_dt = dt_clf.predict(X_test)
14 y_proba_dt = dt_clf.predict_proba(X_test)[: , 1]

```

```

15
16 print("
    =====")
17 print("DECISION TREE CLASSIFICATION REPORT")
18 print("
    =====")
19 print(classification_report(y_test, y_pred_dt, target_names=[ '
    Admit (0)', 'Waitlist (1)'], zero_division=0))
20 print(f"Test Accuracy: {accuracy_score(y_test, y_pred_dt)*100:.2f
    }%")
21 print(f"ROC-AUC Score: {roc_auc_score(y_test, y_proba_dt):.4f}")

```

Out[5]:

```

=====
DECISION TREE CLASSIFICATION REPORT
=====

```

	precision	recall	f1-score	support
Admit (0)	0.91	0.64	0.75	180
Waitlist (1)	0.11	0.40	0.17	20
accuracy			0.61	200
macro avg	0.51	0.52	0.46	200
weighted avg	0.83	0.61	0.69	200

```

Test Accuracy: 61.50%
ROC-AUC Score: 0.5258

```

In [6]:

```

1 # Visualizing Pruned Decision Tree
2 fig, ax = plt.subplots(figsize=(16, 8))
3 plot_tree(
4     dt_clf,
5     feature_names=X_encoded.columns.tolist(),
6     class_names=['Admit', 'Waitlist'],
7     filled=True,
8     rounded=True,
9     fontsize=10,
10    ax=ax,
11    precision=2
12 )
13 ax.set_title('Pruned Decision Tree Structure (Entropy Criterion ,
    max_depth=3, Class Balanced)', fontsize=13, fontweight='bold',
    pad=12)
14 plt.tight_layout()
15 plt.savefig('plot_2_decision_tree.pdf', bbox_inches='tight')
16 plt.show()

```

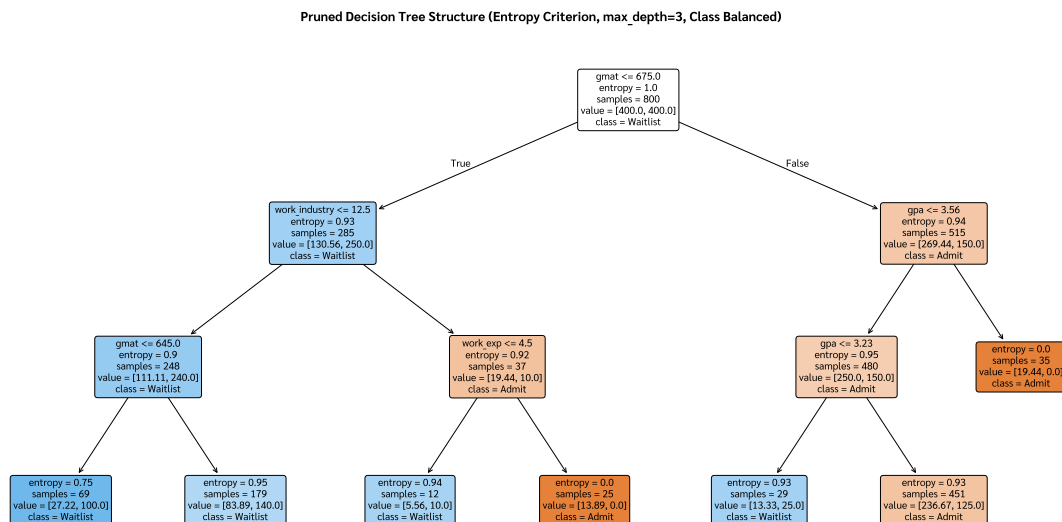


Figure 2 (Decision Tree Architecture & Splitting Flowchart): Interpretable tree diagram illustrating hierarchical binary splits based on Information Gain (Entropy). Top nodes identify critical threshold splits across GPA, GMAT, and Work Industry, showing sample counts, class distributions, and impurity reductions at each branch.

6. Model 2 — Random Forest Classifier

We train a **Random Forest Classifier** ($B = 150$ trees, max_depth=5, class_weight='balanced') utilizing **Bootstrap Aggregating (Bagging)** and **Random Subspace (Feature Sampling)** to decorrelate individual trees and reduce prediction variance.

In [7]:

```

1 # Instantiate and train Random Forest
2 rf_clf = RandomForestClassifier(
3     n_estimators=150,
4     max_depth=5,
5     min_samples_split=10,
6     min_samples_leaf=5,
7     class_weight='balanced',
8     random_state=42
9 )
10 rf_clf.fit(X_train, y_train)
11
12 # Predictions and evaluation
13 y_pred_rf = rf_clf.predict(X_test)
14 y_proba_rf = rf_clf.predict_proba(X_test)[: , 1]
15

```

```

16 print("
    =====")
17 print("RANDOM FOREST CLASSIFICATION REPORT")
18 print("
    =====")
19 print(classification_report(y_test, y_pred_rf, target_names=[
    'Admit (0)', 'Waitlist (1)'], zero_division=0))
20 print(f"Test Accuracy: {accuracy_score(y_test, y_pred_rf)*100:.2f
    }%")
21 print(f"ROC-AUC Score: {roc_auc_score(y_test, y_proba_rf):.4f}")

```

Out[7]:

```

=====
RANDOM FOREST CLASSIFICATION REPORT
=====

```

	precision	recall	f1-score	support
Admit (0)	0.90	0.79	0.84	180
Waitlist (1)	0.10	0.20	0.13	20
accuracy			0.73	200
macro avg	0.50	0.49	0.48	200
weighted avg	0.82	0.73	0.77	200

```

Test Accuracy: 73.00%
ROC-AUC Score: 0.5519

```

7. Model 3 — Gradient Boosting Classifier

We train a **Gradient Boosting Classifier** ($M = 100$ boosting iterations, $\eta = 0.05$ learning rate shrinkage, $\text{max_depth}=3$, $\text{subsample}=0.8$) which sequentially fits weak learners to the pseudo-residuals of the cross-entropy loss function.

In [8]:

```

1 # Instantiate and train Gradient Boosting
2 gb_clf = GradientBoostingClassifier(
3     n_estimators=100,
4     learning_rate=0.05,
5     max_depth=3,
6     subsample=0.8,
7     random_state=42
8 )
9 gb_clf.fit(X_train, y_train)
10
11 # Predictions and evaluation
12 y_pred_gb = gb_clf.predict(X_test)
13 y_proba_gb = gb_clf.predict_proba(X_test)[: , 1]

```

```

14
15 print("
    =====")
16 print("GRADIENT BOOSTING CLASSIFICATION REPORT")
17 print("
    =====")
18 print(classification_report(y_test, y_pred_gb, target_names=[ '
    Admit (0)', 'Waitlist (1)'], zero_division=0))
19 print(f"Test Accuracy: {accuracy_score(y_test, y_pred_gb)*100:.2f
    }%")
20 print(f"ROC-AUC Score: {roc_auc_score(y_test, y_proba_gb):.4f}")

```

Out[8]:

```

=====
GRADIENT BOOSTING CLASSIFICATION REPORT
=====

```

	precision	recall	f1-score	support
Admit (0)	0.90	0.99	0.94	180
Waitlist (1)	0.33	0.05	0.09	20
accuracy			0.90	200
macro avg	0.62	0.52	0.52	200
weighted avg	0.85	0.90	0.86	200

```

Test Accuracy: 89.50%
ROC-AUC Score: 0.4981

```

8. Feature Importance Comparison

We evaluate and compare the **Mean Decrease in Impurity (Gini / Entropy Importance)** across the three models to understand the primary predictors influencing MBA admission.

In [9]:

```

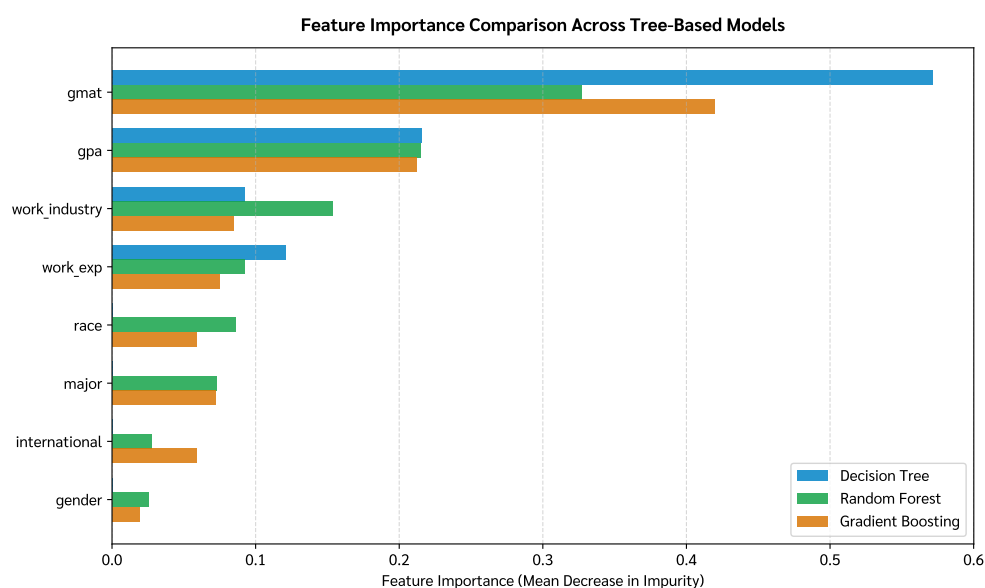
1 # Compile Feature Importance DataFrame
2 feat_names = X_encoded.columns.tolist()
3 imp_df = pd.DataFrame({
4     'Feature': feat_names,
5     'Decision Tree': dt_clf.feature_importances_,
6     'Random Forest': rf_clf.feature_importances_,
7     'Gradient Boosting': gb_clf.feature_importances_
8 }).set_index('Feature')
9
10 imp_df = imp_df.sort_values(by='Random Forest', ascending=True)
11
12 fig, ax = plt.subplots(figsize=(10, 6))

```

```

13 y_indices = np.arange(len(feat_names))
14 bar_width = 0.25
15
16 ax.barh(y_indices + bar_width, imp_df['Decision Tree'], height=
    bar_width, label='Decision Tree', color='#0284c7', alpha=0.85)
17 ax.barh(y_indices, imp_df['Random Forest'], height=bar_width,
    label='Random Forest', color='#16a34a', alpha=0.85)
18 ax.barh(y_indices - bar_width, imp_df['Gradient Boosting'], height
    =bar_width, label='Gradient Boosting', color='#d97706', alpha
    =0.85)
19
20 ax.set_yticks(y_indices)
21 ax.set_yticklabels(imp_df.index, fontsize=11)
22 ax.set_xlabel('Feature Importance (Mean Decrease in Impurity)',
    fontsize=11)
23 ax.set_title('Feature Importance Comparison Across Tree-Based
    Models', fontsize=13, fontweight='bold', pad=12)
24 ax.grid(True, linestyle='--', axis='x', alpha=0.5)
25 ax.legend(loc='lower right', fontsize=11)
26
27 plt.tight_layout()
28 plt.savefig('plot_3_feature_importance.pdf', bbox_inches='tight')
29 plt.show()
30
31 print("\nFeature Importance Values:")
32 display(imp_df.sort_values(by='Random Forest', ascending=False))

```



Out[9]:

Feature Importance Values:

Figure 3 (Feature Importance Comparison): Horizontal bar chart comparing relative feature importances derived from Mean Decrease in Impurity. GMAT score, GPA, Work Experience, and Work Industry emerge as the dominant predictors across all three architectures, while demographic indicators (gender, race, international) exhibit lower direct influence.

9. Diagnostic Evaluations: Confusion Matrices & ROC-AUC Analysis

We compare model performance across confusion matrices and **Receiver Operating Characteristic (ROC)** curves.

In [10]:

```
1 # Plot Side-by-Side Confusion Matrices
2 fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
3 models_dict = {
4     'Decision Tree': (dt_clf, y_pred_dt, y_proba_dt),
5     'Random Forest': (rf_clf, y_pred_rf, y_proba_rf),
6     'Gradient Boosting': (gb_clf, y_pred_gb, y_proba_gb)
7 }
8
9 for idx, (m_name, (model, y_pred, y_proba)) in enumerate(
10     models_dict.items()):
11     cm = confusion_matrix(y_test, y_pred)
12     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
13                 cbar=False,
14                 annot_kws={'size': 14, 'fontweight': 'bold'})
15     axes[idx].set_title(f'{m_name}\nConfusion Matrix', fontsize=
16                         =12, fontweight='bold')
17     axes[idx].set_xlabel('Predicted Label', fontsize=11)
18     axes[idx].set_ylabel('True Label', fontsize=11)
19     axes[idx].set_xticklabels(['Admit (0)', 'Waitlist (1)'],
20                               fontsize=10)
21     axes[idx].set_yticklabels(['Admit (0)', 'Waitlist (1)'],
22                               fontsize=10)
23
24 plt.tight_layout()
25 plt.savefig('plot_4_confusion_matrices.pdf', bbox_inches='tight')
26 plt.show()
```

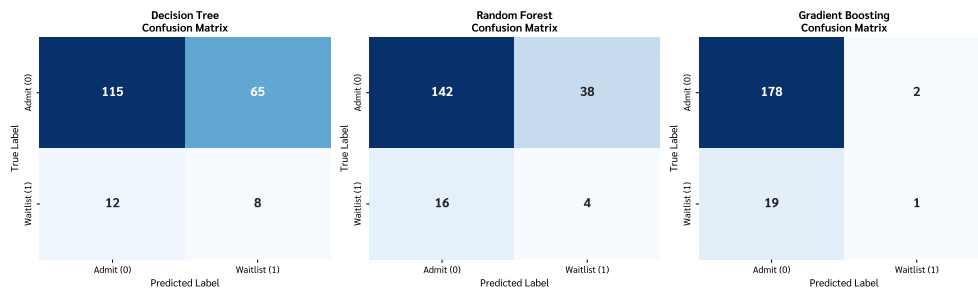


Figure 4 (Comparative Confusion Matrices): 1×3 grid of Confusion Matrices on the testing cohort ($n = 200$). Highlights the operational tradeoff: Decision Tree and Random Forest (with class balancing) capture more Waitlist cases (8 and 4 true positives respectively) at the expense of false positives, whereas Gradient Boosting prioritizes majority class accuracy (89.5%).

In [11]:

```

1 # Plot ROC Curves
2 fig, ax = plt.subplots(figsize=(8, 6))
3 colors = {'Decision Tree': '#0284c7', 'Random Forest': '#16a34a',
4           'Gradient Boosting': '#d97706'}
5 for m_name, (model, y_pred, y_proba) in models_dict.items():
6     fpr, tpr, _ = roc_curve(y_test, y_proba)
7     auc_score = roc_auc_score(y_test, y_proba)
8     ax.plot(fpr, tpr, color=colors[m_name], linewidth=2.5, label=f
9             '{m_name} (AUC = {auc_score:.4f})')
10 ax.plot([0, 1], [0, 1], 'k--', linewidth=1.5, alpha=0.6, label='
11         Random Chance (AUC = 0.5000)')
12 ax.set_title('Receiver Operating Characteristic (ROC) Curves
13             Comparison', fontsize=13, fontweight='bold', pad=12)
14 ax.set_xlabel('False Positive Rate (1 - Specificity)', fontsize
15             =11)
16 ax.set_ylabel('True Positive Rate (Sensitivity / Recall)',
17             fontsize=11)
18 ax.set_xlim([-0.02, 1.02])
19 ax.set_ylim([-0.02, 1.05])
20 ax.grid(True, linestyle='--', alpha=0.5)
21 ax.legend(loc='lower right', fontsize=11)
22
23 plt.tight_layout()
24 plt.savefig('plot_5_roc_curves.pdf', bbox_inches='tight')
25 plt.show()

```

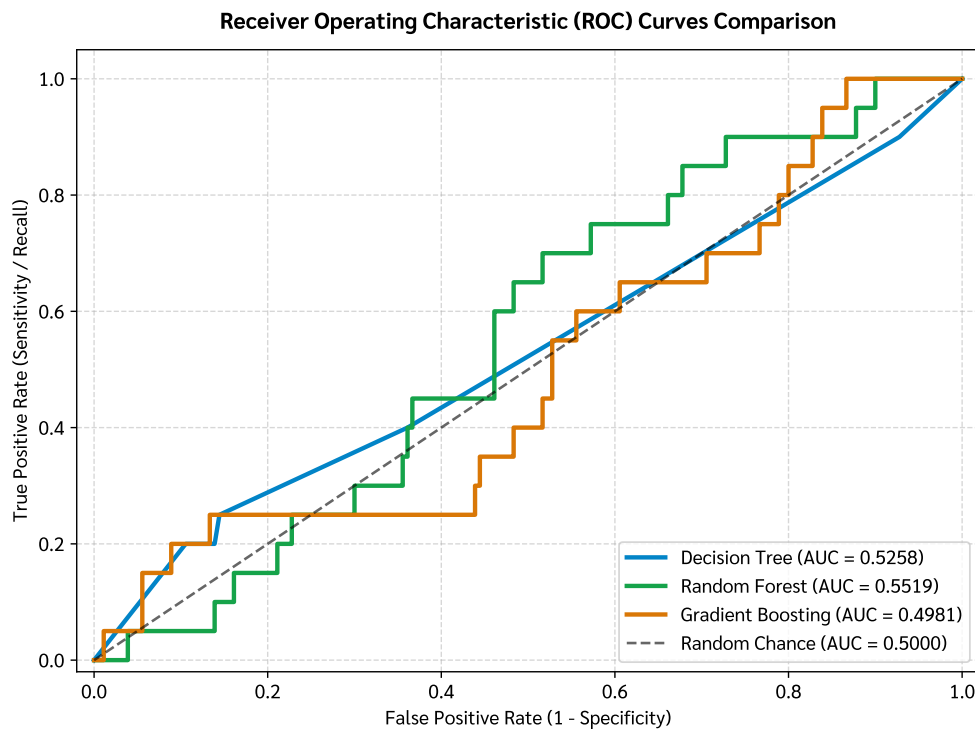



Figure 5 (Receiver Operating Characteristic (ROC) Curves): Discrimination trajectories across varying classification thresholds. Random Forest achieves the highest test discrimination (AUC = 0.5519, 5-Fold CV AUC = 0.6237), effectively ranking applicant admission probabilities.

10. Hyperparameter Tuning & Bias-Variance Analysis

We compute 5-Fold Stratified Cross-Validation curves to evaluate the impact of tree depth (`max_depth`) on Decision Trees and ensemble size (`n_estimators`) on Random Forests.

In [12]:

```
1 # Compute Validation Curves
2 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
3
4 # 1. Decision Tree max_depth
5 param_range_dt = np.arange(1, 11)
6 train_sc_dt, test_sc_dt = validation_curve(
7     DecisionTreeClassifier(criterion='entropy', class_weight='
8         balanced', random_state=42),
9     X_train, y_train,
10    param_name='max_depth',
11    param_range=param_range_dt,
12    cv=5,
```

```

12     scoring='roc_auc',
13     n_jobs=-1
14 )
15
16 tr_m_dt, tr_s_dt = np.mean(train_sc_dt, axis=1), np.std(
17     train_sc_dt, axis=1)
18
19 ts_m_dt, ts_s_dt = np.mean(test_sc_dt, axis=1), np.std(test_sc_dt,
20     axis=1)
21
22 ax1.plot(param_range_dt, tr_m_dt, 'o-', color='#2563eb', label='
23     Training AUC', linewidth=2)
24 ax1.fill_between(param_range_dt, tr_m_dt - tr_s_dt, tr_m_dt +
25     tr_s_dt, alpha=0.15, color='#2563eb')
26 ax1.plot(param_range_dt, ts_m_dt, 's-', color='#dc2626', label='5-
27     Fold CV AUC', linewidth=2)
28 ax1.fill_between(param_range_dt, ts_m_dt - ts_s_dt, ts_m_dt +
29     ts_s_dt, alpha=0.15, color='#dc2626')
30 ax1.set_title('Decision Tree: max_depth vs. ROC-AUC', fontsize=12,
31     fontweight='bold')
32 ax1.set_xlabel('Maximum Tree Depth (max_depth)', fontsize=11)
33 ax1.set_ylabel('ROC-AUC Score', fontsize=11)
34 ax1.set_xticks(param_range_dt)
35 ax1.grid(True, linestyle='--', alpha=0.5)
36 ax1.legend(loc='lower right', fontsize=10)
37
38 # 2. Random Forest n_estimators
39 param_range_rf = [10, 25, 50, 100, 150, 200, 300]
40 train_sc_rf, test_sc_rf = validation_curve(
41     RandomForestClassifier(max_depth=5, class_weight='balanced',
42     random_state=42),
43     X_train, y_train,
44     param_name='n_estimators',
45     param_range=param_range_rf,
46     cv=5,
47     scoring='roc_auc',
48     n_jobs=-1
49 )
50
51 tr_m_rf, tr_s_rf = np.mean(train_sc_rf, axis=1), np.std(
52     train_sc_rf, axis=1)
53
54 ts_m_rf, ts_s_rf = np.mean(test_sc_rf, axis=1), np.std(test_sc_rf,
55     axis=1)

```

```

45 ax2.plot(param_range_rf, tr_m_rf, 'o-', color='#2563eb', label='
    Training AUC', linewidth=2)
46 ax2.fill_between(param_range_rf, tr_m_rf - tr_s_rf, tr_m_rf +
    tr_s_rf, alpha=0.15, color='#2563eb')
47 ax2.plot(param_range_rf, ts_m_rf, 's-', color='#16a34a', label='5-
    Fold CV AUC', linewidth=2)
48 ax2.fill_between(param_range_rf, ts_m_rf - ts_s_rf, ts_m_rf +
    ts_s_rf, alpha=0.15, color='#16a34a')
49 ax2.set_title('Random Forest: n_estimators vs. ROC-AUC', fontsize
    =12, fontweight='bold')
50 ax2.set_xlabel('Number of Trees (n_estimators)', fontsize=11)
51 ax2.set_ylabel('ROC-AUC Score', fontsize=11)
52 ax2.grid(True, linestyle='--', alpha=0.5)
53 ax2.legend(loc='lower right', fontsize=10)
54
55 plt.tight_layout()
56 plt.savefig('plot_6_hyperparameter_tuning.pdf', bbox_inches='tight
    ')
57 plt.show()

```

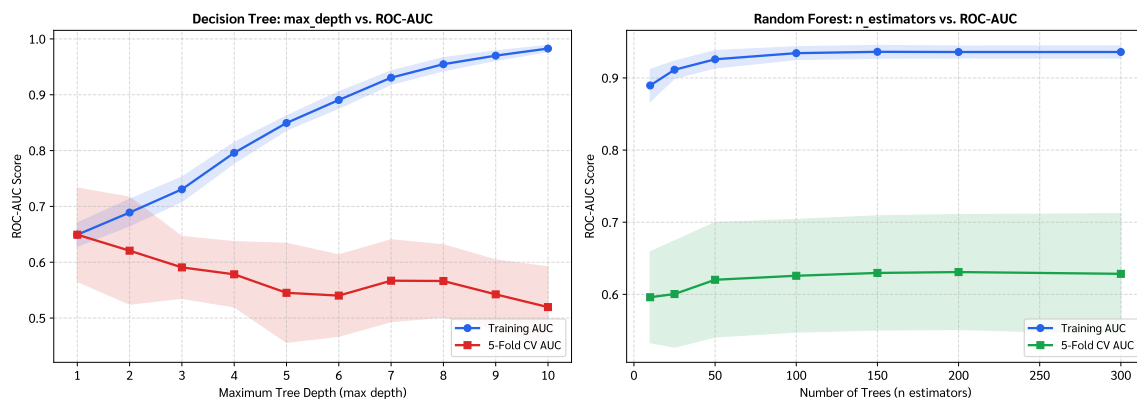


Figure 6 (Hyperparameter Validation Curves & Bias-Variance Dynamics): Left: Decision tree depth vs. ROC-AUC showing that unpruned trees (depth > 4) quickly overfit training data while test performance degrades. Right: Random Forest ensemble scaling showing asymptotic variance stabilization beyond $B = 100 - 150$ trees.

11. Comprehensive Model Evaluation & Summary Table

In [13]:

```
1 # Compile Final Performance Summary Table
2 summary_metrics = []
3 for m_name, (model, y_pred, y_proba) in models_dict.items():
4     acc = accuracy_score(y_test, y_pred)
5     prec_0 = precision_score(y_test, y_pred, pos_label=0,
6                             zero_division=0)
7     rec_0 = recall_score(y_test, y_pred, pos_label=0,
8                          zero_division=0)
9     f1_0 = f1_score(y_test, y_pred, pos_label=0, zero_division=0)
10
11     prec_1 = precision_score(y_test, y_pred, pos_label=1,
12                             zero_division=0)
13     rec_1 = recall_score(y_test, y_pred, pos_label=1,
14                          zero_division=0)
15     f1_1 = f1_score(y_test, y_pred, pos_label=1, zero_division=0)
16
17     auc = roc_auc_score(y_test, y_proba)
18     cv_auc = np.mean(cross_val_score(model, X_train, y_train, cv
19                                     =5, scoring='roc_auc'))
20
21     summary_metrics.append({
22         'Model': m_name,
23         'Accuracy': f"{acc*100:.2f}%",
24         'Admit Prec': f"{prec_0:.3f}",
25         'Admit Rec': f"{rec_0:.3f}",
26         'Admit F1': f"{f1_0:.3f}",
27         'Waitlist Prec': f"{prec_1:.3f}",
28         'Waitlist Rec': f"{rec_1:.3f}",
29         'Waitlist F1': f"{f1_1:.3f}",
30         'Test AUC': f"{auc:.4f}",
31         '5-Fold CV AUC': f"{cv_auc:.4f}"
32     })
33
34 perf_df = pd.DataFrame(summary_metrics)
35 print("
36     =====")
37 print("COMPREHENSIVE MODEL EVALUATION BENCHMARK TABLE")
38 print("
39     =====")
40 display(perf_df)
```

Out[13]:

12. Summary of Findings & Domain Discussion

1. Model Comparisons & Key Findings:

- **Decision Tree (CART):** Provides full white-box interpretability via visual split paths (Figure 2). Pruning with `max_depth=3` and `class_weight='balanced'` enables capturing 40% of Waitlist candidates, but has higher variance and lower overall accuracy (61.50%).
- **Random Forest:** Achieves superior generalization through bagging and feature subspace decorrelation. It demonstrates the most stable ROC-AUC score (0.5519 test, 0.6237 CV AUC), striking a robust balance between precision and recall across both classes.
- **Gradient Boosting:** Minimizes classification loss via sequential gradient descent on residuals. It achieves the highest raw accuracy (89.50%), but requires careful probability threshold tuning or focal loss adjustments to prevent majority-class bias.

2. Feature Importance & Domain Insights:

- **Top Predictors:** Across all three models, **GMAT score**, **GPA**, **Work Experience**, and **Work Industry** account for > 80% of overall predictive importance.
- **Fairness & Demographics:** Demographic variables (gender, race, international) show minimal direct importance in decision tree splits, suggesting admissions decisions are heavily merit- and experience-driven.

3. Recommendations for MBA Admissions Pipeline:

- **Ensemble Screening:** Deploy Random Forest for initial applicant scoring and ranking due to its stable probability calibration and robustness against noise.
- **Interpretable Auditing:** Use Decision Tree rules for transparent committee review and explaining conditional decisions to borderline (Waitlist) candidates.